Prudentia AI Operator Manual

Last updated: 2026-09-09

This manual explains how to run the local Prudentia AI deployment. It is written for the person responsible for Docker Desktop, Ollama, model caches, startup, shutdown, updates, backup, restore, and runtime troubleshooting.

For in-app administration, use the Administrator Manual (administrator-manual.md). For day-to-day usage, use the End User Manual (end-user-manual.md).

1. Runtime Overview

The standard offline Windows deployment uses:

- Windows with Docker Desktop running Linux containers.
- Docker Compose for Prudentia AI services.
- A host-installed Ollama service for chat and embedding models.
- Containers reaching Ollama at `http://host.docker.internal:11434`.
- Runtime model downloads disabled by airgap environment flags.
- vLLM services disabled until the `vllm` Compose profile is deliberately used.

The default frontend URL is:

```
http://localhost:3000
```

The API is reached through the frontend reverse proxy:

```
http://localhost:3000/api/v1
```

Citation protocol rollout

Normal chat uses single-pass generation and local quote matching. Adaptive and Never faithfulness policies make no citation judge or answer-repair calls. Strict (Always), forced internal verification, and artifact verification retain model-based support checks with the configured deadline and bounded repair.

Distinguish `source_matched` from semantic `verified` in logs and evaluations. An `unverified` local result preserves the answer with incomplete-match labeling; source-access and revision failures still block delivery. Measure time to first text and total generation separately from optional semantic verification. Provider connection failures can still prevent generation in either mode.

Previously indexed documents may lack original-text provenance. Reingest selected documents explicitly to populate native/OCR/generated span classifications and revision identity; this upgrade does not automatically reingest the corpus. Missing caches still fail under the airgap policy. Do not download models during a query to make verification succeed.

`source_revision_unavailable` means the cited generation, file or evidence no longer matches, or the evidence is no longer accessible. Investigate the source's lifecycle and ingestion state without substituting a newer file into an old citation. Historic file retention is unchanged. An

`artifact_citation_verification_failed` job has exhausted its single citation repair and ends without publishing unsupported files; queue redelivery does not repeat that repair. A user can explicitly retry after the underlying issue is resolved.

2. Prerequisites

On the Windows host:

- Docker Desktop is installed and switched to Linux containers.
- WSL 2 backend is enabled.
- Ollama is installed, running, and has the selected chat and embedding models.
- The project source bundle, Docker image tar, and cache tar files have been transferred to the machine.
- Enough disk is available for Docker images, volumes, document storage, and model caches. Keep tens of GB free before adding vLLM.

Before disconnecting a preparation machine from the internet, download required installers for Docker Desktop, WSL updates, GPU drivers, Ollama, and internal security software.

3. Required Bundle Files

An offline bundle should include at least:

- `AgenticRAG-source.zip`
- `.env.windows-airgap`
- `agenticrag-runtime-images.tar`
- `backend-reranker-cache.tgz`
- `backend-docling-cache.tgz`

If vLLM is being prepared later, the vLLM image and model cache are handled as a separate rollout. Do not enable vLLM during an Ollama-only deployment.

4. Runtime Environment File

The offline Windows deployment should use `.env.windows-airgap`. The most important values are:

```
FRONTEND_PORT=3000
BOOTSTRAP_ADMIN_EMAIL=admin@prudentia.ai
BOOTSTRAP_ADMIN_PASSWORD=replace-with-local-admin-password
RAG_MODEL_PROVIDER=ollama
OLLAMA_BASE_URL=http://host.docker.internal:11434
OLLAMA_CHAT_MODEL=llama3.1:8b
OLLAMA_EMBED_MODEL=nomic-embed-text:latest
VITE_API_BASE_URL=
AIRGAP_RUNTIME_OFFLINE=1
HF_HUB_OFFLINE=1
TRANSFORMERS_OFFLINE=1
HF_DATASETS_OFFLINE=1
CLAMAV_SCAN_ENABLED=true
```

Set the Ollama model names to exactly match `ollama list` on the Windows host. Leave `VITE_API_BASE_URL` blank for the default same-origin setup.

The general `.env.example` and the Windows air-gap example both bootstrap with `RAG_MODEL_PROVIDER=ollama`. If an administrator has already saved a workspace RAG configuration, that saved record remains active until **Restore deployment defaults** is used in Runtime Settings.

5. First Installation On Offline Windows

Open PowerShell in the copied bundle directory and run:

```
Set-ExecutionPolicy -Scope Process Bypass
.\deploy_windows.ps1
```

If `C:\AgenticRAG\AgenticRAG` already exists and you want to replace the source tree while keeping Docker data volumes, run:

```
.\deploy_windows.ps1 -ReplaceSource
```

The deployment script imports the source, images, and model cache folders used by the runtime.

6. Manual Installation Checks

If you are installing manually, load Docker images:

```
docker load --input "$Bundle\agenticrag-runtime-images.tar"
docker image ls
```

Create and import cache folders:

```
New-Item -ItemType Directory -Force .\model-cache\fastembed | Out-Null
New-Item -ItemType Directory -Force .\model-cache\docling | Out-Null
tar -xzf "$Bundle\backend-reranker-cache.tgz" -C .\model-cache\fastembed
tar -xzf "$Bundle\backend-docling-cache.tgz" -C .\model-cache\docling
```

Do not import `postgres-data`, `qdrant-data`, `minio-data`, `neo4j-data`, or `redis-data` for a fresh install. Docker Compose creates them. Import data volumes only when restoring or migrating an existing deployment.

7. Verify Ollama

Run these commands on the Windows host:

```
ollama list
Invoke-RestMethod http://localhost:11434/api/tags
```

Confirm that `.env.windows-airgap` uses installed model names.

If containers cannot reach Ollama through `host.docker.internal`, configure Ollama to listen on an address Docker Desktop can reach, commonly `OLLAMA_HOST=0.0.0.0:11434`, then restart Ollama. Restrict access with Windows Firewall because Ollama should not be exposed to untrusted networks.

8. Start The Stack

From the project root:

```
docker compose --env-file .env.windows-airgap up -d --no-build --pull never --wait
```

Open:

```
http://localhost:3000
```

The first Platform Admin signs in with the bootstrap account from the environment file and must change the bootstrap password immediately.

9. Health Checks

Use these checks after startup:

```
docker compose --env-file .env.windows-airgap ps
Invoke-RestMethod http://localhost:3000/healthz
$ApiHealth = "http://127.0.0.1:8000/health/live"
docker compose --env-file .env.windows-airgap exec api `
  python -c "import urllib.request as u;print(u.urlopen('$ApiHealth').status)"
docker compose --env-file .env.windows-airgap logs --tail 100 api
docker compose --env-file .env.windows-airgap logs --tail 100 ingestion-worker
```

Check container-to-host Ollama connectivity:

```
docker compose --env-file .env.windows-airgap exec api `
  python -c "import socket;socket.create_connection(('host.docker.internal',11434),5)"
```

10. Stop And Restart

Stop the stack without deleting data:

```
docker compose --env-file .env.windows-airgap down
```

Start it again:

```
docker compose --env-file .env.windows-airgap up -d --no-build --pull never --wait
```

Do not run `docker compose down -v` unless you intentionally want to delete application data and caches.

11. Update An Offline Host

Prepare a new bundle on a connected machine, then on the offline host:

```
docker compose --env-file .env.windows-airgap down
docker load --input "$Bundle\agenticrag-runtime-images.tar"
```

Replace the source directory with the new release source, keep `.env.windows-airgap`, and start again:

```
docker compose --env-file .env.windows-airgap up -d --no-build --pull never --wait
```

Do not delete data volumes unless the update explicitly requires a clean install.

12. Data Volumes

Expected named volumes include:

Volume	Contains
`agenticrag_postgres-data`	Application database.
`agenticrag_qdrant-data`	Vector database.
`agenticrag_minio-data`	Uploaded source files and generated artifacts.
`agenticrag_redis-data`	Redis append-only data.
`agenticrag_neo4j-data`	GraphRAG graph database data.
`agenticrag_vllm-model-cache`	vLLM model cache, only needed when vLLM is enabled.

Expected host cache folders include:

Folder	Contains
`model-cache/fastembed`	FastEmbed sparse, dense, and reranker cache.
`model-cache/docling`	Docling and RapidOCR artifacts.
`model-cache/vllm`	vLLM Hugging Face cache when vLLM is enabled.

13. Cold Backup

Use a cold backup whenever possible. Stop the stack first so service volumes are not changing while they are archived.

```
docker compose --env-file .env.windows-airgap down

$Backup = "D:\prudentia-backup-$(Get-Date -Format yyyyMMdd-HHmmss)"
New-Item -ItemType Directory -Force $Backup | Out-Null

$Volumes = @(
  "postgres-data",
  "qdrant-data",
  "minio-data",
  "redis-data",
  "neo4j-data"
)

foreach ($Volume in $Volumes) {
  docker run --rm `
    -v "agenticrag_$Volume:/from" `
    -v "${Backup}:/backup" `
    postgres:16-alpine `
    sh -c "cd /from && tar -czf /backup/$Volume.tgz ."
}

Compress-Archive -Path .\.env.windows-airgap -DestinationPath "$Backup\env-file.zip"
```

Store the backup in a protected location. The backup contains sensitive documents, credentials, generated artifacts, chat history, and audit data.

14. Restore From Backup

Restore only to a stopped stack.

```
docker compose --env-file .env.windows-airgap down

$Backup = "D:\prudentia-backup-YYYYMMDD-HHMMSS"
$Volumes = @(
  "postgres-data",
  "qdrant-data",
  "minio-data",
  "redis-data",
  "neo4j-data"
)

foreach ($Volume in $Volumes) {
  docker volume create "agenticrag_$Volume" | Out-Null
  docker run --rm `
    -v "agenticrag_$Volume:/to" `
    -v "${Backup}:/backup" `
    postgres:16-alpine `
    sh -c "cd /to && tar -xzf /backup/$Volume.tgz"
}

docker compose --env-file .env.windows-airgap up -d --no-build --pull never --wait
```

Restore the matching `.env.windows-airgap` when recovering a full deployment. Secrets, provider choices, ports, and model names must match the restored data.

15. ClamAV And Uploads

Uploads may fail if virus scanning is enabled but the ClamAV database is not available. For a pilot-only run, an operator may set:

```
CLAMAV_SCAN_ENABLED=false
```

For production, ship a ClamAV image with the approved database or use an internal mirror/update process.

16. vLLM Later

Do not enable vLLM for the Ollama-only deployment. vLLM services start only when the `vllm` Compose profile is enabled.

Before switching the app provider to vLLM:

1. Load or build the vLLM image.
2. Import the vLLM model cache.
3. Verify Docker Desktop GPU support.
4. Start the required vLLM services.
5. Confirm endpoints are healthy.
6. Change provider in **Runtime Settings** or environment.
7. Recreate API and worker services.

Switching provider to vLLM before services and caches are ready leaves the app pointing at unavailable endpoints.

17. Common Runtime Problems

Problem	Check	Fix
Compose tries to pull or build offline	Command missing `--no-build --pull never` or image missing	Load the runtime image tar and restart with offline flags.
App opens but login fails	API, Postgres, Redis, bootstrap account	Check `docker compose ps`, API logs, and user state.
Chat fails	`RAG_MODEL_PROVIDER`, Ollama URL, model names, Ollama reachability	Verify Ollama host service and container-to-host API tags request.
Upload fails immediately	ClamAV, file type, file size, MinIO	Check ClamAV status, upload limits, and API logs.
PDF ingestion fails offline	Docling or OCR cache missing	Run Docling verify-only command from ingestion worker.
PPTX ingestion reports renderer missing	LibreOffice Impress is absent from the document-pipeline image	Rebuild the ingestion-worker image and verify `soffice --version` inside it.
PPTX visual analysis fails	Slide render, vision endpoint, or model unavailable	Check ingestion-worker logs, LibreOffice, and the configured vision model endpoint.
Reranking fails	FastEmbed cache missing	Run FastEmbed verify-only command from API container.
Jobs stay queued	Ingestion worker not healthy or Redis unavailable	Check `ingestion-worker` logs and Redis health.
GraphRAG unavailable	`graphrag-worker` or Neo4j unhealthy	Check `graphrag-worker`, `neo4j`, and Ingestion Health.
Live DB asks for a database	Routing metadata is missing or multiple catalogs match	Review each approved catalog's description, entities, metrics, examples, and exclusions.
Live DB query fails	Connector target unreachable, generated SQL rejected, or catalog mismatch	Check source-routing and `sql_generation`, `sql_validation`, and `sql_execution` timings; verify network route, credentials, approved catalog, and timeout.

Docling cache verification:

```
docker compose --env-file .env.windows-airgap exec ingestion-worker `
  python -m rag.ingestion.ops.prewarm_docling --verify-only
```

FastEmbed cache verification:

```
docker compose --env-file .env.windows-airgap exec api `
  python -m rag.ops.prewarm_fastembed --verify-only --all-rerankers
```

18. Operator Safety Rules

- Do not delete Docker volumes unless a clean install is intended.
- Do not change secrets casually on a deployment with existing data.
- Do not switch model providers without verifying model endpoints and caches.
- Do not expose Ollama, Postgres, Redis, MinIO, Qdrant, Neo4j, or Docker APIs to untrusted networks.

- Keep backup media protected because it contains sensitive source files and generated artifacts.
- Record incidents with timestamps, commands run, service status, and relevant log excerpts.